

Luca_favoriti.R

Utente

2026-06-06

```
rm(list=ls());gc();

##          used (Mb) gc trigger (Mb) max used (Mb)
## Ncells  584158 31.2   1306722 69.8   928635 49.6
## Vcells 1112661  8.5    8388608 64.0  1928164 14.8

setwd("C:/Users/Utente/OneDrive/Universita/Magistrale/2025-2026/Aziendali/Progetto/Analisi")

load("dati_puliti.Rdata")

source("C:/Users/Utente/OneDrive/Universita/Magistrale/2025-2026/Aziendali/Labs/Lab7/utils.R")

## Warning: il pacchetto 'dplyr' è stato creato con R versione 4.5.3
##
## Caricamento pacchetto: 'dplyr'
## I seguenti oggetti sono mascherati da 'package:stats':
##
##   filter, lag
## I seguenti oggetti sono mascherati da 'package:base':
##
##   intersect, setdiff, setequal, union
## Warning: il pacchetto 'partykit' è stato creato con R versione 4.5.3
## Caricamento del pacchetto richiesto: grid
## Caricamento del pacchetto richiesto: libcoin
## Warning: il pacchetto 'libcoin' è stato creato con R versione 4.5.3
## Caricamento del pacchetto richiesto: mvtnorm
## Warning: il pacchetto 'mvtnorm' è stato creato con R versione 4.5.3
MSE <- function(previsioni, veri_valori){
  mean( (previsioni - veri_valori)^2 )
}

plot_tree <- function(albero, file_name = NULL){
  # Se viene fornito un nome file, attiva il dispositivo corretto
  if(!is.null(file_name)){
    png(file = paste0(file_name, ".png"), width = 1800, height = 1200, res = 200)
  }

  # Logica del grafico (immutata)
```

```

plot(albero, col = "gray40", lwd = 1.8)
text(albero, pretty = 4, digits = 3, cex = 0.8, col = "darkblue", font = 2)

if(!is.null(file_name)) {
  dev.off()
}
dev.off()
}

dataset_regressione_favoriti <-
dataset_regressione_favoriti |>
rename(Google_Trends = avg_interest_global_last_year,
       Log_Review_Count = Log_Seller_Reviews_Count,
       Item_Verification = Has_Item_Verification)

nomi_presentazione <- c(
  "Size" = "Taglia",
  "Condition" = "Condizioni dell'Articolo",
  "Material" = "Materiale",
  "Favorites_Count" = "Numero di Preferiti",
  "Shipping_Cost" = "Costo di Spedizione",
  "Color_new" = "Colore",
  "Seller_Rating_Class" = "Fascia Recensione Venditore",
  "Log_Review_Count" = "Log(N° Recensioni)",
  "Seller_has_distintivi" = "Venditore con Badge",
  "Num_Other_Items" = "Altri Articoli in Vendita",
  "Google_Trends" = "Interesse (Google Trends)"
)

# Regressione sui favoriti ----

stima <- dataset_regressione_favoriti[id_stima,]
verifica <- dataset_regressione_favoriti[id_verifica,]

X_stima <- model.matrix(y~., stima)[,-1] |> as.data.frame()
X_verifica <- model.matrix(y~., verifica)[,-1] |> as.data.frame()

set.seed(1)
idx_a <- sample(1:nrow(X_stima), ceiling(NROW(X_stima)*2/3))
idx_b <- setdiff(1:nrow(X_stima), idx_a)

## CART ----

library(tree)
m_tree_full <- tree(y~., data=stima,
                  split='deviance',
                  control=
                    tree.control(nobs=NROW(stima),
                                minsize=1,
                                mindev=0.0001))
sum(m_tree_full$frame$var == "<leaf>")

```

```
## [1] 473
```

```

max_size <- 50
K_cv <- 20
set.seed(1)
fold <- sample(1:K_cv, NROW(stima), replace = T)
matrice_errori_albero = matrix(nrow=K_cv, ncol=max_size)
rownames(matrice_errori_albero) <- paste("fold", 1:K_cv)
#for (j in 1:K_cv){
#  id_stima = which(fold != j)
#  id_verifica = which(fold == j)
#
#  m_tree_tmp = tree(y~.,
#                    data=stima[id_stima,],
#                    split='deviance',
#                    control=
#                      tree.control(nobs=NROW(stima),
#                                    minsize=1,
#                                    mindev=0.0001))
#  m_tree_tmp
#
#  tree_list = lapply(2:(max_size+1), # per noi dopo i 50 split é sovra-adattato
#                    # non uso il numero di split m_tree_full perché può
#                    # essere che alcuni alberi siano più piccoli
#                    function(l) prune.tree(m_tree_tmp,
#                                             best=l) )
#  pred_list = lapply(tree_list,
#                    function(x) predict(x,
#                                         stima[id_verifica,]))
#  lista_errori_CI = lapply(pred_list,
#                    function(x) MSE(x, stima$y[id_verifica]))
#
#  matrice_errori_albero[j,] = lista_errori_CI |> unlist()
#}
#save(matrice_errori_albero, file="Modelli_stimati/err_tree_favoriti.RData")
load("Modelli_stimati/err_tree_favoriti.RData")
dim(matrice_errori_albero)

## [1] 20 50

errore_albero_CI = apply(matrice_errori_albero, 2, mean)
se_albero <- apply(matrice_errori_albero, 2, function(x) sd(x) / sqrt(K_cv))

B = which.min(errore_albero_CI)

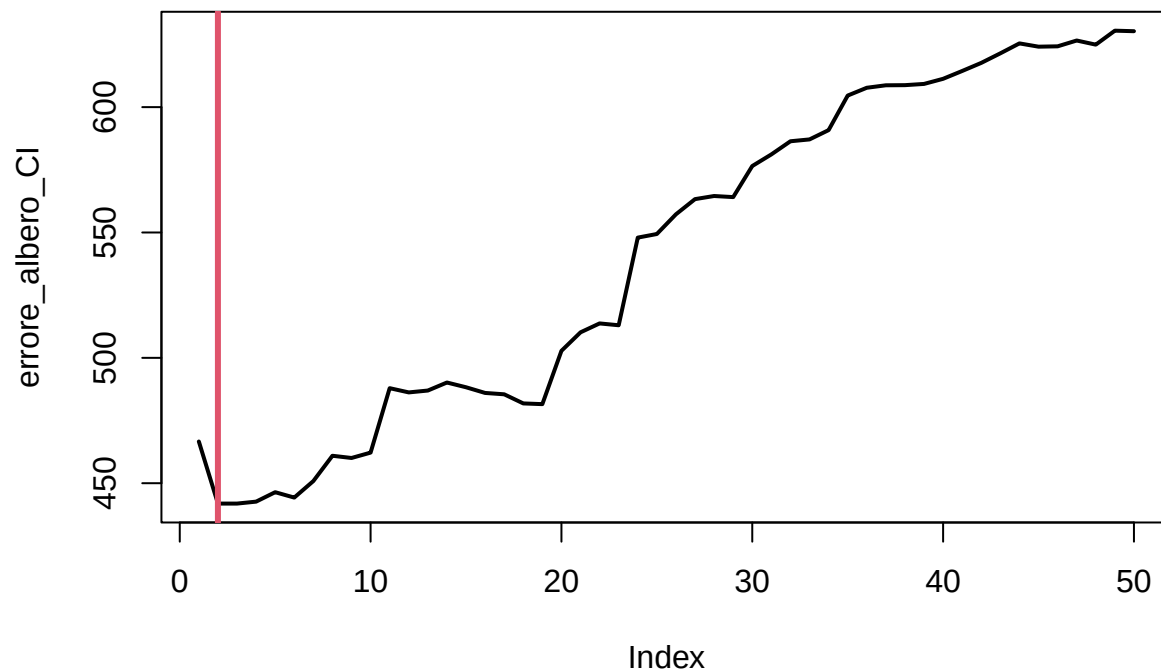
cv_min <- errore_albero_CI[B]
se_min <- se_albero[B]

soglia_1se <- cv_min + se_min

taglie_valide <- which(errore_albero_CI <= soglia_1se)
B_1se <- max(2, min(taglie_valide))

plot(errore_albero_CI, type='l', lwd=2)
abline(v=B, col=2, lwd=3)

```



```

m_tree = prune.tree(m_tree_full, best=B_1se)

plot_tree(m_tree, "Plot/albero_fav")

## null device
##      1

yhat_tree <- predict(m_tree, newdata = verifica)
err_tree <- mean((verifica$y - yhat_tree)^2)

fav_medio_verifica <- mean(verifica$y)
devianza_totale_verifica <- sum((verifica$y - fav_medio_verifica)^2)
devianza_residua_albero <- sum((verifica$y - yhat_tree)^2)

r2_tree <- 1 - (devianza_residua_albero / devianza_totale_verifica)

r2_fav <- data.frame(albero = r2_tree)
r2_fav

##      albero
## 1 0.04038074

rmse_albero <- sqrt(mean((verifica$y - yhat_tree)^2))
rmse_fav <- data.frame(albero = rmse_albero)
rmse_fav

##      albero
## 1 20.78664

mae_albero <- mean(abs(verifica$y - yhat_tree))
mae_fav <- data.frame(albero = mae_albero)
mae_fav

##      albero
## 1 14.00929

```

```

## Random forest -----

n_threads <- parallel::detectCores()
library(ranger)

## Warning: il pacchetto 'ranger' è stato creato con R versione 4.5.3
NCOL(stima) |> sqrt(); NCOL(stima) |> log();

## [1] 3.872983
## [1] 2.70805

mtry_values <- c(2,4,6,8,10)
num_tree_values <- c(50, 100, 200, 300, 400)
griddina <- expand.grid(mtry=mtry_values,
                      n_tree=num_tree_values)

modelli_rf <- list()
set.seed(1)
for(i in 1:NROW(griddina)){
  cat(i, "su", NROW(griddina), "\n\n")
  modelli_rf[[i]] <- ranger(y~., num.trees = griddina[i,2],
                          mtry=griddina[i,1],
                          data=stima, importance="none",
                          num.threads=n_threads)
}

## 1 su 25
##
## 2 su 25
##
## 3 su 25
##
## 4 su 25
##
## 5 su 25
##
## 6 su 25
##
## 7 su 25
##
## 8 su 25
##
## 9 su 25
##
## 10 su 25
##
## 11 su 25
##
## 12 su 25
##
## 13 su 25
##
## 14 su 25
##
## 15 su 25

```

```

##
## 16 su 25
##
## 17 su 25
##
## 18 su 25
##
## 19 su 25
##
## 20 su 25
##
## 21 su 25
##
## 22 su 25
##
## 23 su 25
##
## 24 su 25
##
## 25 su 25

#save(modelli_rf, file="Modelli_stimati/err_rf_fav.RData")
load("Modelli_stimati/err_rf_fav.RData")

errori_OOB <- vapply(modelli_rf, function(x) x$prediction.error, FUN.VALUE = numeric(1))
datini <- data.frame(n_tree = griddina$n_tree,
                     mtry = as.factor(griddina$mtry), errore_OOB = errori_OOB)

library(ggplot2)
ggplot(datini, aes(x = n_tree, y = errore_OOB, color = mtry, group = mtry)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_color_viridis_d(option = "viridis", end = 0.85) +
  scale_x_continuous(breaks = num_tree_values) +
  theme_minimal(base_size = 11) +
  labs(
    title = "Tuning Random Forest: Errore Out-of-Bag",
    x = "Numero di alberi",
    y = "Errore OOB",
    color = "Variabili\\ncandidate"
  ) +
  theme(plot.title = element_text(face = "bold", size = 13),
        panel.grid.minor = element_blank())

ntree_rf <- 300
set.seed(123)
rf_best <- ranger(y~.,
                  num.trees = ntree_rf, mtry = 2,
                  data=stima, importance="permutation",
                  num.threads=n_threads)

p_rf = predict(rf_best, verifica)
devianza_residua_rf <- sum((verifica$y - p_rf$predictions)^2)

```

```

r2_rf <- 1 - (devianza_residua_rf / devianza_totale_verifica)

r2_fav$RF <- r2_rf
r2_fav

##      albero      RF
## 1 0.04038074 0.1606916

rmse_rf <- sqrt(mean((verifica$y - p_rf$predictions)^2))
rmse_fav$RF <- rmse_rf
rmse_fav

##      albero      RF
## 1 20.78664 19.43997

mae_rf <- mean(abs(verifica$y - p_rf$predictions))
mae_fav$RF <- mae_rf
mae_fav

##      albero      RF
## 1 14.00929 13.13788

### Importanza ----

top_imp <- sort(rf_best$variable.importance, decreasing = TRUE) |> head(10)

tipo_var <- function(nomi) {
  # Classifico manualmente le variabili in 3 gruppi semantici
  dplyr::case_when(
    grepl("Seller|Rating|Reviews|distintivi|Num_Other", nomi) ~ "Venditore",
    grepl("Brand|Size|Condition|Material|Color|Trends", nomi) ~ "Prodotto",
    TRUE ~ "Transazione"
  )
}

df_imp <- data.frame(
  variabile = names(top_imp),
  importanza = as.numeric(top_imp)
) |>
  mutate(
    variabile = factor(variabile, levels = rev(variabile)),
    tipo = tipo_var(as.character(variabile))
  )

p_imp_rf <- df_imp %>%
  mutate(
    Nome_Bello = coalesce(nomi_presentazione[variabile], variabile),
    Nome_Bello = forcats::fct_reorder(Nome_Bello, importanza)
  ) |>
  ggplot(aes(x = Nome_Bello, y = importanza, fill = tipo)) +
  geom_col(width = 0.7) +
  coord_flip() +
  scale_fill_manual(
    values = c("Venditore" = "#4E79A7", "Prodotto" = "#F28E2B",
              "Transazione" = "#59A14F"),

```

```

    name = "Categoria"
  ) +
  labs(title = "Importanza delle top 10 variabili tramite permutazione",
        x = NULL, y = "Importanza") +
  theme_minimal(base_size = 12) +
  theme(plot.title = element_text(face = "bold", size = 14),
        legend.position = "bottom")

ggsave("Plot/Importanza_RF_fav.png", plot=p_imp_rf,
       width = 8, height = 5)

```

GBM -----

```

nus <- c(0.001, 0.005, 0.01) # shrinkage
dps <- c(8, 11, 14, 17, 20) # profondità

```

```

tuning_par <- expand.grid(nus, dps)
err_gbm <- rep(1, nrow(tuning_par))

```

```
library(gbm)
```

Warning: il pacchetto 'gbm' è stato creato con R versione 4.5.3

Loaded gbm 2.2.3

This version of gbm is no longer under development. Consider transitioning to gbm3, <https://github.com>

```

nts_xgb <- 4000
set.seed(123)
# modello di prova per capire se uso un numero corretto di alberi
mod_prova <- gbm(stima$y~., data=X_stima,
                 distribution = "gaussian", n.trees = nts_xgb,
                 shrinkage = tuning_par[1,1],
                 interaction.depth = tuning_par[1,2])

yhat_mp <- predict(mod_prova, newdata = X_verifica, n.trees=1:nts_xgb)
err_pr <- apply(yhat_mp, 2, function(p) mean((verifica$y - p)^2))
plot(err_pr, type="l")
abline(h=min(err_pr), col=2)
abline(v=which.min(err_pr), col=2, lty=2, lwd=2)
abline(v=3000, col=3, lty=2, lwd=2)

err_pr[450] / err_pr[100]

```

```

##      450
## 0.9262823

```

```
err_pr[which.min(err_pr)] / err_pr[100]
```

```

##      2519
## 0.847752

```

```
nts_xgb <- 2750
```

Tuning ----


```

#set.seed(1)
#for(i in 1:nrow(tuning_par)){
#  mod <- gbm(stima$y[idx_a]~., data=X_stima[idx_a,],
#            distribution = "gaussian",
#            shrinkage = tuning_par[i,1], n.trees = nts_xgb,
#            interaction.depth = tuning_par[i,2])
#  passo <- max(30, 1000 * (0.5 - tuning_par[i,1])/(dps/2))
#  alberi_pred <- seq(100, nts_xgb, by = passo)
#  ps <- predict(mod, newdata = X_stima[idx_b,], n.trees = alberi_pred)
#  err_i <- apply(ps, 2, function(p) mean(((stima$y)[idx_b] - p)^2))
#  err_gbm[i] <- min(err_i)
#  cat(round(i/nrow(tuning_par)*100,2), "%\n")
#}
#save(err_gbm, file="Modelli_stimati/err_gbm_fav.RData")
load("Modelli_stimati/err_gbm_fav.RData")
fav_medio_idxb <- mean(stima$y[idx_b])
devianza_totale_idxb <- sum((stima$y[idx_b] - fav_medio_idxb)^2)

err_gbm <- 1 - (err_gbm*length(idx_b)) / devianza_totale_idxb

zmat <- matrix(err_gbm, nrow = length(nus), ncol = length(dps), byrow = TRUE)

par_opt <- tuning_par[which.max(err_gbm),] # un parametro di shrinkage troppo elevato mi fa
par_opt

```

```

##      Var1 Var2
## 8 0.005   14

```

```

cbind(tuning_par, err_gbm)

```

```

##      Var1 Var2   err_gbm
## 1 0.001    8 0.2416945
## 2 0.005    8 0.2404165
## 3 0.010    8 0.2356253
## 4 0.001   11 0.2387028
## 5 0.005   11 0.2402258
## 6 0.010   11 0.2416669
## 7 0.001   14 0.2384070
## 8 0.005   14 0.2422528
## 9 0.010   14 0.2362741
## 10 0.001  17 0.2394558
## 11 0.005  17 0.2386806
## 12 0.010  17 0.2368210
## 13 0.001  20 0.2403558
## 14 0.005  20 0.2406985
## 15 0.010  20 0.2333513

```

```

risultati_tuning <- data.frame(tuning_par)
colnames(risultati_tuning) <- c("shrinkage", "depth")
risultati_tuning$err_gbm <- err_gbm # Aggiungiamo l'errore calcolato nel ciclo

```

```

library(ggplot2)
library(viridis)

```

```

## Warning: il pacchetto 'viridis' è stato creato con R versione 4.5.2

```

```
## Caricamento del pacchetto richiesto: viridisLite

ggplot(risultati_tuning, aes(x = factor(depth), y = factor(shrinkage))) +
  geom_tile(aes(fill = err_gbm), color = "white", linewidth = 0.2) +
  geom_tile(data = risultati_tuning[which.max(risultati_tuning$err_gbm), ],
            aes(x = factor(depth), y = factor(shrinkage)),
            fill = NA, color = "black", linewidth = 1.2) +
  scale_fill_viridis_c(option = "plasma", direction = 1, name = "R2") +
  labs(
    title = "Tuning dei Parametri GBM",
    subtitle = "Il riquadro nero indica la combinazione con l'R2 minimo",
    x = "Profondità",
    y = "Shrinkage"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", size = 14),
    plot.subtitle = element_text(color = "gray40", size = 10),
    axis.text = element_text(size = 10),
    panel.grid = element_blank()
  )
ggsave("Plot/tuning_gbm_fav.png")
```

```
## Saving 7 x 7 in image
```

```
### Fit finale ----
```

```
set.seed(123)
mod_gbm <- gbm(stima$y[idx_a]~., data=X_stima[idx_a,],
               n.trees = nts_xgb, distribution = "gaussian",
               shrinkage = par_opt[1],
               interaction.depth = par_opt[2])

ps <- predict(mod_gbm, newdata = X_stima[idx_b,],
              n.trees = seq(60, nts_xgb, by=20))
err_i <- apply(ps, 2, function(p) mean((stima$y[idx_b] - p)^2))
n_trees_best <- names(err_i)[which.min(err_i)] |> as.numeric()
n_trees_best
```

```
## [1] 540
```

```
yhat_gbm <- predict(mod_gbm, newdata = X_verifica,
                   n.trees = n_trees_best)
devianza_residua_gbm <- sum((verifica$y - yhat_gbm)^2)

r2_gbm <- 1 - (devianza_residua_gbm / devianza_totale_verifica)

r2_fav$GBM <- r2_gbm
r2_fav
```

```
##      albero      RF      GBM
## 1 0.04038074 0.1606916 0.1816312
```

```
rmse_gbm <- sqrt(mean((verifica$y - yhat_gbm)^2))
rmse_fav$gbm <- rmse_gbm
rmse_fav
```

```

##      albero      RF      gbm
## 1 20.78664 19.43997 19.19594

mae_gbm <- mean(abs(verifica$y - yhat_gbm))
mae_fav$gbm <- mae_gbm
mae_fav

##      albero      RF      gbm
## 1 14.00929 13.13788 12.76669

colnames(X_stima)

## [1] "LogPrice"
## [2] "Brandasics"
## [3] "Brandautry"
## [4] "BrandMarche di Lusso"
## [5] "Brandchampion"
## [6] "Brandconverse"
## [7] "Branddiadora"
## [8] "Brandfila"
## [9] "Brandignoto"
## [10] "Brandnew balance"
## [11] "Brandnike"
## [12] "Brandpuma"
## [13] "Brandreebok"
## [14] "Brandvans"
## [15] "BrandAltro"
## [16] "Size"
## [17] "Condition.L"
## [18] "Condition.Q"
## [19] "Condition.C"
## [20] "Condition^4"
## [21] "Materialnoto"
## [22] "Shipping_Cost"
## [23] "Is_BoostedTRUE"
## [24] "Item_VerificationTRUE"
## [25] "Color_newColorato"
## [26] "Color_newMulticolor"
## [27] "Color_newScuro"
## [28] "Seller_Rating_ClassBasso (<= 4.0)"
## [29] "Seller_Rating_ClassBuono (4.1 - 4.6)"
## [30] "Seller_Rating_ClassOttimo (4.7 - 4.9)"
## [31] "Seller_Rating_ClassPerfetto (5.0)"
## [32] "Log_Review_Count"
## [33] "Seller_has_distintivi"
## [34] "Num_Other_Items"
## [35] "Google_Trends"

dim(X_stima)

## [1] 1434 35

# Trovo le top3 variabili per devianza spiegata
top3_gbm <- summary(mod_gbm, n.trees = n_trees_best,
                    plotit = T, order = TRUE)$var[1:3]
# Indici colonna delle top 3 variabili
idx_top3 <- sapply(top3_gbm, function(v) which(colnames(X_stima) == v))

```

```

# Definisco la funzione per calcolare i dati del Partial Dependence Plot singolo
pdp_gbm_single <- function(var_idx, modello, X_data, n_trees, dati_orig,
                             n_grid = 50) {
  # Estraggo il nome della variabile, i dati e verifico se è numerica
  var_name <- colnames(X_data)[var_idx]
  x <- X_data[[var_name]]
  is_num <- is.numeric(x)

  # Creo la griglia: sequenza per i numerici (escludendo le code 2%-98%), categorie uniche altrimenti
  griglia <- if (is_num)
    seq(quantile(x, 0.02), quantile(x, 0.98), length.out = n_grid)
  else
    sort(unique(x))

  # Calcolo la predizione media per ogni valore della griglia
  pds <- sapply(griglia, function(v) {
    tmp <- X_data
    tmp[[var_name]] <- v
    mean(predict(modello, newdata = tmp, n.trees = n_trees))
  })

  # Restituisco i risultati impaginati in un dataframe
  data.frame(x = griglia, pd = pds, variabile = var_name, is_num = is_num)
}

# Calcolo i dati PDP per le top 3 variabili applicando la funzione
pdp_gbm_list <- lapply(idx_top3, pdp_gbm_single,
                        modello = mod_gbm,
                        X_data = X_stima,
                        n_trees = n_trees_best,
                        dati_orig = stima)

# Genero la lista dei 3 grafici ggplot
pdp_gbm_plots <- lapply(pdp_gbm_list, function(df) {

  nome_brutto <- df$variabile[1]
  nome_bello <- coalesce(nomi_presentazione[nome_brutto], nome_brutto)

  # Inizializzo il grafico gestendo l'asse x in base al tipo di variabile
  p <- ggplot(df, aes(x = if (df$is_num[1]) as.numeric(x) else factor(x),
                     y = pd))

  # Aggiungo la geometria adeguata: linea se numerico, barre se categorico
  if (df$is_num[1]) {
    p <- p + geom_line(color = "#E15759", linewidth = 1.2)
  } else {
    p <- p + geom_col(fill = "#E15759", width = 0.6)
  }

  # Formatto i titoli e il tema estetico
  p + labs(title = nome_bello, x = NULL, y = "Prezzo atteso (€)") +
    theme_minimal(base_size = 11) +
    theme(plot.title = element_text(face = "bold"))
})

```

```

})

library(patchwork)

## Warning: il pacchetto 'patchwork' è stato creato con R versione 4.5.3
# Unisco i tre grafici affiancati e aggiungo un titolo globale
p_pdp_gbm <- (pdp_gbm_plots[[1]] | pdp_gbm_plots[[2]] | pdp_gbm_plots[[3]]) +
  plot_annotation(
    theme = theme(plot.title = element_text(face = "bold", size = 14))
  )

ggsave("Plot/gbm_pdp_top3_fav.png",
       plot = p_pdp_gbm, width = 12, height = 5, dpi = 200)

## Warning: Use of `df$is_num` is discouraged.
## i Use `is_num` instead.

## Warning: Use of `df$is_num` is discouraged.
## i Use `is_num` instead.
## Use of `df$is_num` is discouraged.
## i Use `is_num` instead.

## Effetti casuali -----

### MERT -----

form <- formula(lm(y~.,
                   data=stima |> dplyr::select(-Brand,
                                              -Condition)))

set.seed(123)
mert_model <- MERT(formula = form , random = " + (1|Brand) + (1|Condition)",
                  data = stima, cv=T)

## Caricamento del pacchetto richiesto: rpart
## Warning: il pacchetto 'rpart' è stato creato con R versione 4.5.3
## Caricamento del pacchetto richiesto: lme4
## Warning: il pacchetto 'lme4' è stato creato con R versione 4.5.2
## Caricamento del pacchetto richiesto: Matrix
## Warning: il pacchetto 'Matrix' è stato creato con R versione 4.5.3

## [1] "Log likelihood: -6410.37054037629"
## [1] "Log likelihood: -6429.15799672749"

summary(mert_model$EffectModel)

## Linear mixed model fit by maximum likelihood ['lmerMod']
## Formula: resid ~ -1 + (1 | Brand) + (1 | Condition)
## Data: newdata
## Control: lmer.control
##
##      AIC      BIC    logLik -2*log(L)  df.resid
## 12864.3 12880.1 -6429.2 12858.3    1431
##

```

```

## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.7669 -0.5772 -0.2777  0.2569  9.1047
##
## Random effects:
##   Groups      Name      Variance Std.Dev.
##   Brand      (Intercept)  61.2029  7.8232
##   Condition  (Intercept)   0.1977  0.4446
##   Residual                    447.4940 21.1541
## Number of obs: 1434, groups: Brand, 15; Condition, 5

#plot_tree(mert_model$Tree, "Plot/MERT_fav")

pred_tree <- predict(mert_model$Tree, newdata = verifica)
pred_ran <- predict(mert_model$EffectModel, newdata = verifica)
pred_mert <- pred_tree + pred_ran

devianza_residua_mert <- sum((verifica$y - pred_mert)^2)

r2_mert <- 1 - (devianza_residua_mert / devianza_totale_verifica)

r2_fav$mert <- r2_mert
r2_fav

##      albero      RF      GBM      mert
## 1 0.04038074 0.1606916 0.1816312 0.02116549

rmse_mert <- sqrt(mean((verifica$y - pred_mert)^2))
rmse_fav$mert <- rmse_mert
rmse_fav

##      albero      RF      gbm      mert
## 1 20.78664 19.43997 19.19594 20.99372

mae_mert <- mean(abs(verifica$y - pred_mert))
mae_fav$mert <- mae_mert
mae_fav

##      albero      RF      gbm      mert
## 1 14.00929 13.13788 12.76669 13.86835

### MERF -----

form <- formula(lm(y~.,
                  data=stima |> dplyr::select(-Brand)))

set.seed(123)
merf_model <- MERF_ranger_safe(formula = form, data = stima,
                              random = " + (1|Brand)",
                              num.trees_final = ntree_rf, # stesso di prima
                              num.threads = n_threads-2,
                              mtry_grid = c(2,4,6,8)
)
merf_model$mtry_last

## [1] 2

```

```

pred_fix <- predict(merf_model$RandomForest, data = verifica)$predictions
pred_ran <- predict(merf_model$EffectModel, newdata = verifica) - predict(merf_model$EffectModel, newda
pred_merf <- pred_fix + pred_ran

devianza_residua_merf <- sum((verifica$y - pred_merf)^2)
r2_merf <- 1 - (devianza_residua_merf / devianza_totale_verifica)

r2_fav$merf <- r2_merf
r2_fav

```

```

##      albero      RF      GBM      mert      merf
## 1 0.04038074 0.1606916 0.1816312 0.02116549 0.173461

```

```

rmse_merf <- sqrt(mean((verifica$y - pred_merf)^2))
rmse_fav$merf <- rmse_merf
rmse_fav

```

```

##      albero      RF      gbm      mert      merf
## 1 20.78664 19.43997 19.19594 20.99372 19.29152

```

```

mae_merf <- mean(abs(verifica$y - pred_merf))
mae_fav$merf <- mae_merf
mae_fav

```

```

##      albero      RF      gbm      mert      merf
## 1 14.00929 13.13788 12.76669 13.86835 12.94126

```

```

### Metboost -----

```

```

#x_var <- colnames(stima |> dplyr::select(-y, -Brand))
#set.seed(1)
#par_opt # del GBM classico
#met_model <- metboost_fit_path_manual(stima, y_name = "y", vars_x = x_var,
#                                     group_var = "Brand",
#                                     #M_max = nts_xgb,
#                                     M_max = 300,
#                                     shrinkage = 0.01, depth = 4)
# group_var effetti casuali
# M_max numero albero, prima ne avevamo 6000, ora per motivi computazionali scendiamo a 300
# inoltre ad ogni step devo stimare un albero con una certa profondità, più depth
# è elevato più ci metterà, quindi metto 2
# lo shrinkage era ottimo a 0.05, ma sappiamo che deve andare assieme al numero di
# alberi e alla profondità. Quindi, dato che fisso 2 come profondità, una scelta per
# bilanciare è di alzare il parametro di shrinkage

#save(met_model, file="met_model.Rdata")
#load("met_model.Rdata")
#
#met_model$lmes[[10]]@optinfo$conv$lme4$messages
#
#pdp_land_sf <- metboost_pdp(fit = met_model, data_ref = stima,
#                             var_name = "Size", n_grid = 40)
#plot(pdp_land_sf$x, pdp_land_sf$y, type="l", xlab = "Size",
#      ylab = "y", col = "blue")
#pdp_cond <- metboost_pdp(fit = met_model, data_ref = stima,
#                          var_name = "Condition")

```

```

#plot(pdp_cond$x, pdp_cond$y, pch=19)
#
#pred_met <- metboost_predict_manual(met_model, verifica)
#err_met <- mean((verifica$y - pred_met$pred)^2)
#r2_fav$metboost <- err_met
#r2_fav

cbind(t(r2_fav), t(rmse_fav), t(mae_fav))

##           [,1]      [,2]      [,3]
## albero 0.04038074 20.78664 14.00929
## RF      0.16069162 19.43997 13.13788
## GBM      0.18163124 19.19594 12.76669
## mert     0.02116549 20.99372 13.86835
## merf     0.17346102 19.29152 12.94126

```